NetSentry: Technical Architecture & Implementation Design

Team: Data_Sentinel

Smart India Hackathon 2026 — Problem Statement 26155

1 Core Philosophy: The Canonical Model

Current network compliance tools rely on fragile, vendor-specific regex pattern libraries that break upon firmware updates or new hardware acquisitions. NetSentry resolves this structural limitation by operating in the space of *security semantics* rather than vendor syntax. The architecture translates heterogeneous configurations from all supported vendors into a vendor-neutral parameter namespace called the **Canonical Security Baseline Model**. All compliance evaluations execute strictly against this canonical representation, meaning vendor identity is irrelevant beyond the parsing stage.

2 System Architecture & Data Pipeline

The system executes a fully deterministic normalization and evaluation pipeline consisting of three primary stages: Ingestion and Identification, Parsing and Normalization, and Compliance Evaluation.

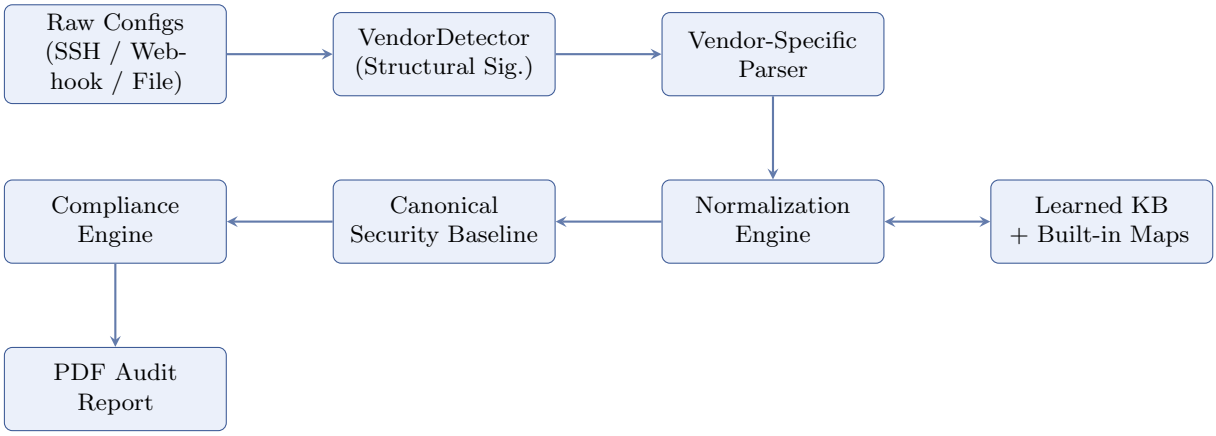


Figure 1: NetSentry: Deterministic Normalization and Evaluation Pipeline

2.1 Ingestion and Identification

Configurations enter via three concurrent paths: manual bulk archive uploads, live SSH extraction using read-only vendor-specific command sequences, and Git webhook integration for continuous infrastructure-as-code compliance monitoring. The **VendorDetector** analyzes structural signatures (e.g., `## Last changed:` for Juniper, `build-version:` for Fortinet) rather than keyword searches, producing a confidence score between 0.92–0.95 for known vendors.

2.2 Parsing and Normalization

Vendor-specific parsers process the configuration while maintaining section hierarchy and context. Parsed lines enter the **Normalization Engine**, which evaluates them against a two-priority mapping stack:

- **Learned Knowledge Base:** Site-specific mappings submitted via the Training Interface, applied first to give precedence to locally trained rules.
- **Built-In Mapping Library:** Pre-compiled regex patterns covering standard security-relevant CLI syntax for nine supported vendors (Cisco IOS/IOS-XE, Juniper Junos, Fortinet FortiOS, Palo Alto PAN-OS, Arista EOS, SONiC, Cumulus Linux, Check Point Gaia, Sophos SFOS).

Every match produces a canonical parameter (e.g., `security.ssh.version = 2`). Full provenance—including verbatim source text, line number, mapping source label, and confidence score—is retained through to the final report.

3 The Compliance Engine

The Compliance Engine evaluates rule logic directly against the Canonical Security Baseline Model.

- **Strict Evaluation:** Supported operators include `equals`, `not_equals`, `contains`, `greater_or_equal`, and `enabled`. Type-aware casting ensures no fuzzy string matching occurs on integer or boolean values.
- **Tri-State Findings:** Results are graded as PASS, FAIL, or NOT_CONFIGURED. The explicit absence of a required

parameter (e.g., no session timeout configured on any VTY line) is treated as a security finding—not a neutral state.
- **Multi-Framework Coverage:** A single canonical evaluation pass simultaneously produces compliance verdicts against CIS Benchmarks, NIST SP 800-53 Rev. 5, DISA STIGs, and ISO/IEC 27001:2022 Annex A, without redundant computation or separate passes.

4 The Dual-Mode AI Engine

NetSentry implements a Strategy Pattern to manage two `BaseAIProvider` implementations, guaranteeing output determinism while enabling optional LLM-backed enhancement:

- **Deterministic Provider (Default):** A pure Python offline heuristic engine using structural token analysis and pre-built security lexicons. It processes configuration files exceeding 20,000 lines in under 200 milliseconds, with no external API dependency or GPU requirement. For each failed control, it generates a structured, evidence-anchored impact narrative—for example, detailing credential interception risks and lateral movement prerequisites when `security.telnet.enabled` evaluates to TRUE. Output is bit-for-bit reproducible for identical inputs.
- **Gemini Provider (Optional):** An administrator may configure a Gemini API key to enable LLM-backed reasoning for complex or ambiguous findings. The provider switch is explicit and fully auditable; the deterministic engine remains available as the fallback at all times.

5 The Training Interface

Traditional compliance tools require vendor-issued software updates to support new CLI syntax—creating a perpetual dependency on the tool vendor's release cycle. NetSentry eliminates this dependency architecturally.

When the Normalization Engine encounters a security-relevant command matching no existing mapping, it adds the command to an unknown-candidates queue rather than discarding it. The Training Interface presents these candidates in a structured review workflow: the administrator views the raw command text, section context, and vendor label; selects the target canonical parameter; specifies the value data type; and optionally provides a custom extraction pattern. On submission, the mapping is written directly to the Learned Knowledge Base and takes effect system-wide immediately, with no restart or code modification required. Each subsequent audit report identifies findings evaluated against trained mappings via the `LEARNED_KNOWLEDGE_BASE` provenance label, providing a full audit trail.

6 Evidence-Based Reporting Engine

The reporting stage converts the evaluated Canonical Security Baseline Model into a factual PDF audit document populated entirely from device-specific data.

- **Automated Device Identification:** Hostname, hardware model, and OS version are extracted directly from the configuration (e.g., via `! Current Configuration` headers on Cisco IOS).
- **Quantitative Risk Score:** Findings are weighted by severity category (CRITICAL, HIGH, MEDIUM, LOW) to produce a 0-to-100 risk index and a corresponding posture label.
- **Verbatim Evidence Citations:** Every finding entry includes the exact source line, line number, and expected versus detected value. Remediation commands are device-specific vendor CLI, not generic guidance.

7 Technical Stack

Component	Technology & Notes
Backend API	`FastAPI` (Python), 12 modular API routers
Database	`SQLite`, seeded with compliance rules and base vendor mappings at first launch
Frontend	`React` + `TypeScript` (Vite), server-side state managed via REST
PDF Engine	Custom rendering pipeline producing evidence-bound audit documents
Deployment	Fully on-premise; single-command launch via provided shell scripts; no cloud dependency